```
end

obj = Special.new("Hello", "there", "World")

puts "Before: obj = #{obj}"
data = obj.to_json
new_obj = Special.from_json(data)
puts "After: obj = #{new_obj}"
```

produces:

```
Before: obj = Hello there World
After: obj = Hello  World
```

Notice that the new object no longer has the volatile attribute, since that was not marshaled.

Distributed Ruby

Because we can serialize an object or a set of objects into a form suitable for out-of-process storage, we can transmit objects from one process to another. Couple this capability with the power of networking, and *voilà*—you have a distributed object system. To save you the trouble of having to write the code, we suggest using Masatoshi Seki's Distributed Ruby library, often abbreviated drb or dRuby, which is available as a standard Ruby library at https://github.com/ruby/drb.

Using drb, a Ruby process may act as a server, as a client, or as both. A drb server acts as a source of objects, while a client is a user of those objects. To the client, it appears that the objects are local, but in reality the code is still being executed remotely.

A server starts a service by associating an object with a given port. Threads are created internally to handle incoming requests on that port, so remember to join the drb thread before exiting your program:

```
puts File.readlines "code/ospace/drb/drb_server.rb"
puts "DRb.thread.join   # Don't exit just yet!"
```

A simple drb client simply creates a local drb object and associates it with the object on the remote server; the local object is a proxy:

ospace/drb/drb_client.rb

```
require "drb"
DRb.start_service
obj = DRbObject.new(nil, "druby://localhost:9000")
# Now use obj
puts "Sum is: #{obj.add(1, 2, 3)}"
```

The client connects to the server and calls the method add, which uses the magic of inject to sum its arguments. It returns the result, which the client prints out:

```
require "code/ospace/drb/drb_server.rb"
require "code/ospace/drb/drb_client.rb"
sleep 1
```

produces:

```
Sum is: 6
```

The initial nil argument to DRbObject indicates that we want to attach to a new distributed object. We could also use an existing object.